1. Introduction

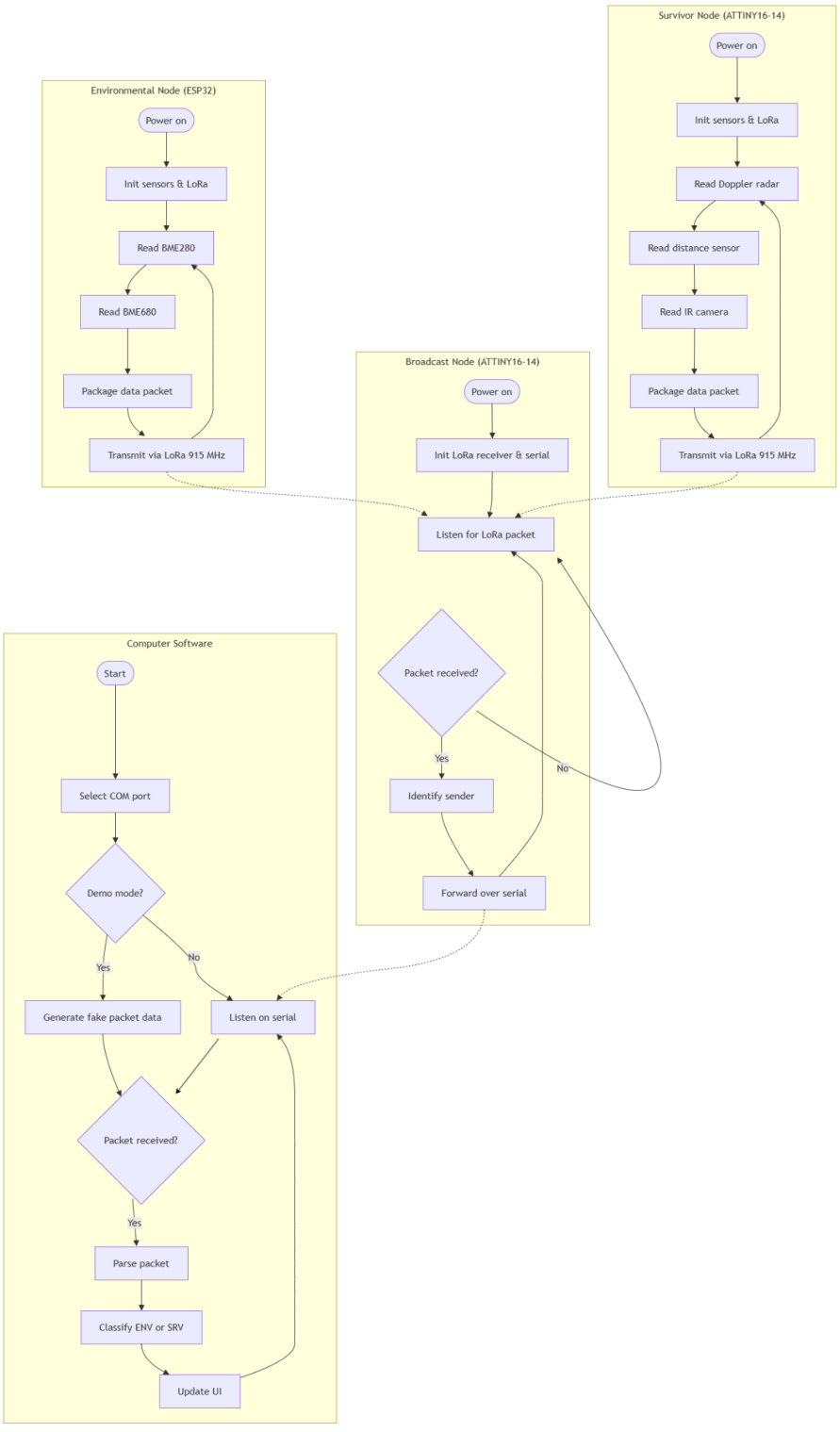
Abstract:
The FirstResponderSystem is an affordable LoRa based sensor network that allows first responders (police, fire, medical) to quickly find survivors and monitor their environment and hazards without relying on spotty or weak Wi-Fi or cell infrastructure.

The Problem:
During major disasters, first responders do not have a reliable way to track their environment, find survivors, and receive information in real time. A report from the Federal Communications Commission (FCC) during 2018 showed that natural disasters like Hurricanes Irma and Maria took down over 95% of cell infrastructure, affecting vital systems such as 911 call centers (Federal Communications Commission, 2018). Existing systems can be expensive, hard to deploy, or require additional personnel to operate, wasting manpower that can be used from more urgent tasks.

The Solution:
The FirstResponderSystem uses inexpensive, durable, and easy-to-deploy custom-made PCBs that create a centralized long-range radio network suited for emergencies instead of relying on broken or buggy connection systems (WiFi, Mobile Data, etc). LoRa has been shown to maintain good connectivity around ~3km in dense, suburban areas with an antenna mounted only a few meters off the ground (Augustin et al., 2016). By providing real-time sensor data and survivor detection, FirstResponderSystem allows emergency personnel to analyze dangerous conditions, locate any survivors, and make safe decisions while allowing responders to work on more urgent things. The data can also be sent to other organizations to relay information to people in the area, declare evacuations/emergencies, and more. The survivor node can be put in rubble to detect survivors, while the environmental node can be put in various parts of the disaster area. Multiple survivor and environmental nodes can be used at once.

Reverie Hacks 2026 Submission - Embedded Systems:
https://github.com/aneeshlingala/FirstResponderSystem

Node Architecture Diagram:



2. System Architecture

PCB and Software Interaction:
FirstResponderSystem runs as a centralized sensor network with three different hardware nodes and computer software to analyze the data. The system utilizes LoRa radios to transceive information without relying on WiFi or cell towers which may fail.

1. Survivor Node (SRV) and Environmental Node (ENV): The Survivor Node and Environmental Node operate by themselves with a battery. Multiple Survivor and Environmental nodes can be deployed in the disaster region. Survivor Nodes can be placed into rubble where survivors may be trapped, while Environmental Nodes can be deployed anywhere with an open area. Each node gets sensor readings, interprets and analyzes the data, makes a data packet, and transmits the data over the 915mHz band.

2. Broadcast Node: The Broadcast Node connects to a computer over Micro-USB to power itself and send data. Only one Broadcast Node can be deployed. It utilizes a LoRa radio transceiver to receive data packets from other independent nodes and uses a serial to USB converter to send data to the computer it is connected to.

3. Computer Software: The computer software acts as the layer between the data packets and human-readable information. It listens to serial packets sent by the Broadcast Node on a COM port, parses every packet and classifies it as SRV or ENV with the node's randomly-generated identification number. It maintains a list of alerts and information from each node to be able to view the readings. The software also includes a "DEMO" mode, where the user interface is populated with fake nodes to show the functionalities of the software and let the users get acquainted with the interface.

Node Requirements:
Survivor and Environmental Nodes are required to be in range of the broadcast node to properly work. The computer software can alert if a node is having connection problems. New nodes can be added without

needing to configure anything, allowing for quick deployment of more nodes if required.

3. Communication Specifications

Radio Specs:
All three nodes utilize the RFM95W LoRa transceiver (receiver + transmitter) with the Semtech SX1276 chipset. The LoRa radio technology was chosen because it had low power usage and long range when compared to WiFi or cellular.

- Frequency: 915 mHz in ISM band
- Transmit Power: +2 dBm to +20 dBm
- Spreading Factor (higher - longer range, slower transmission, vice versa): SF9
- Expected Range: 5-20km (depends on tuning)
- Data Rate: ~1.8 kbps
- Antenna: u.FL connector to attach an antenna
- Survivor Node Packet Format:
  `RSSI:<rssi>,SNR:<snr>,DATA:ID:<node_id>,X:<x>,Y:<y>,Z:<z>,M:<motion>,P:<presence>,D:<distance>,H:<human>`
- Environmental Node Packet Format:
  `RSSI:<rssi>,SNR:<snr>,DATA:Node ID:<node_id>,Temp:<temp>,Pressure:<pressure>,Humidity:<humidity>,X gyro:<x>,Y gyro:<y>,Z gyro:<z>,Gas temp:<gas_temp>,VOC:<voc>`

4. Software

Computer Software Info:
The computer software (client) is built for Python 3.11+ and utilizes Tkinter for the GUI part of the client. It uses pyserial to interface with the COM ports and receive serial data. It can be used without needing to install dependencies on Windows computers when the .exe is run. The .exe was built with py2exe. For usage on other operating systems, the .py file can be

used. However, there needs to be Python 3.11+, Tkinter (provided with most Python software distributions), and the pyserial module installed.

Serial Reader:
A background process called serial_reader opens the selected COM port at 115200 baud rate. Each line from the serial input is pushed onto an info queue and is refreshed to show on the screen every 300ms.

Packet Parsing:
Each line utilizes a comma delimitated format. The software is able to differentiate node type by the start of the packet, where it either says it is under ENV (environmental) or SRV (survivor). This keeps the data format more human-readable when debugging is required.

Node Tracking:
Each node's sensor data is saved in memory, allowing for a rolling history of sensor outputs. The sensor data is either shown in text or in a live chart to the user. Nodes are either in the online state, stale state (no packet in 15sec), or offline state (no packet in 60sec). These switching states allow for capturing and reading accurate data and alerts.

Alerts:
The software has three varying levels of alerts: info, warning, and critical. These alerts can be triggered by human presence detection, RSSI (signal strength) dropping below -95dBm, or a node going stale or offline.

Demo Mode:
Demo mode runs a thread that simulates realistic data packets for various simulated nodes. The data packets are populated with randomly-generated info, allowing for demos of the software without having nodes on hand.

5. PCB Design

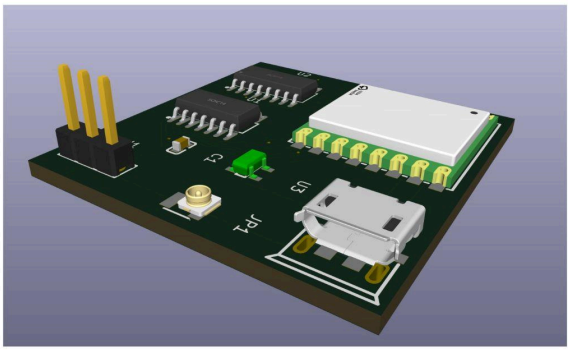
Details:
All of the three nodes were custom-designed in KiCAD 10 as a double layer board. The footprints and symbols used in the PCBs were from LCSC, already bundled in KiCAD, or RCWL/SnapEDA.

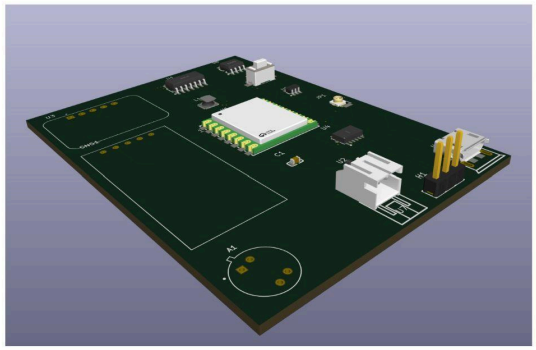
Survivor Node PCB:

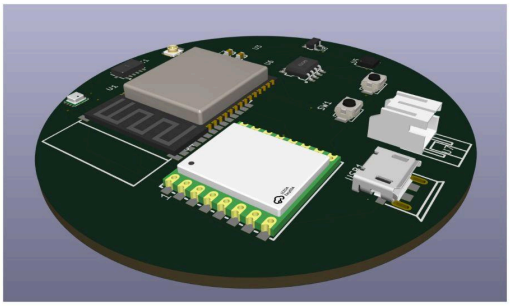
- MCU: ATTINY16-14
- Key Components: Doppler radar, distance sensor, IR camera, and RFM95W LoRa transceiver
- Size: 75 x 56mm
- Power: Battery-powered Li-ion
- Cost: $11-12 per PCB + parts cost

Environmental Node PCB:

- MCU: ESP32
- Key components: BME280, BME680, and RFM95W LoRa transceiver
- Size: Circular - 62.17 x 62.17mm
- Power: Battery-powered Li-ion
- Cost: $7-8 per PCB + parts cost







Broadcast Node PCB:

- MCU: ATTINY16-14
- Key components: RFM95W LoRa transceiver, and Micro-USB port
- Size: 34.76 x 35.98mm
- Power: Micro-USB Port
- Cost: $5.50-7.00 per PCB + parts cost

Design Considerations:

For the MCU, the ATTINY16-14 was chosen for the Survivor Node because it was a chip that could last long on batteries while still being able to get proper data from its sensors. For the environmental node, the ESP32 was chosen because it had better compatibility and power for more complex sensors, while still conserving battery. For the Broadcast Node, the ATTINY16-14 was chosen because the components and purpose of the PCB were simple

6. Testing

Status:

As of now, this project has been digitally tested and validated in every phase that doesn't require PCB fabrication.

What has been validated:

- Schematic and layout was tested using KiCAD's electrical rules checker
- PCB was tested using KiCAD's design rules checker
- Computer software was tested by sending packets matching the expected format over serial
- Firmware code was able to compile and could fit under the flash storage limit of the MCU

What needs to be done:

- Fabrication and assembly of the PCBS (either by hand-soldering or PCBA)
- LoRa range testing
- Packet reliability over varying distances
- Battery life testing

7. Sustainability and Scalability

Cost and Deployment:
Each PCB is very inexpensive to make (about $1-2 per PCB - excluding parts), making FirstResponderSystem much affordable to deploy at scale compared to other similar systems like SAR equipment or thermal drones. Nodes are also redeployable if they aren't too damaged.

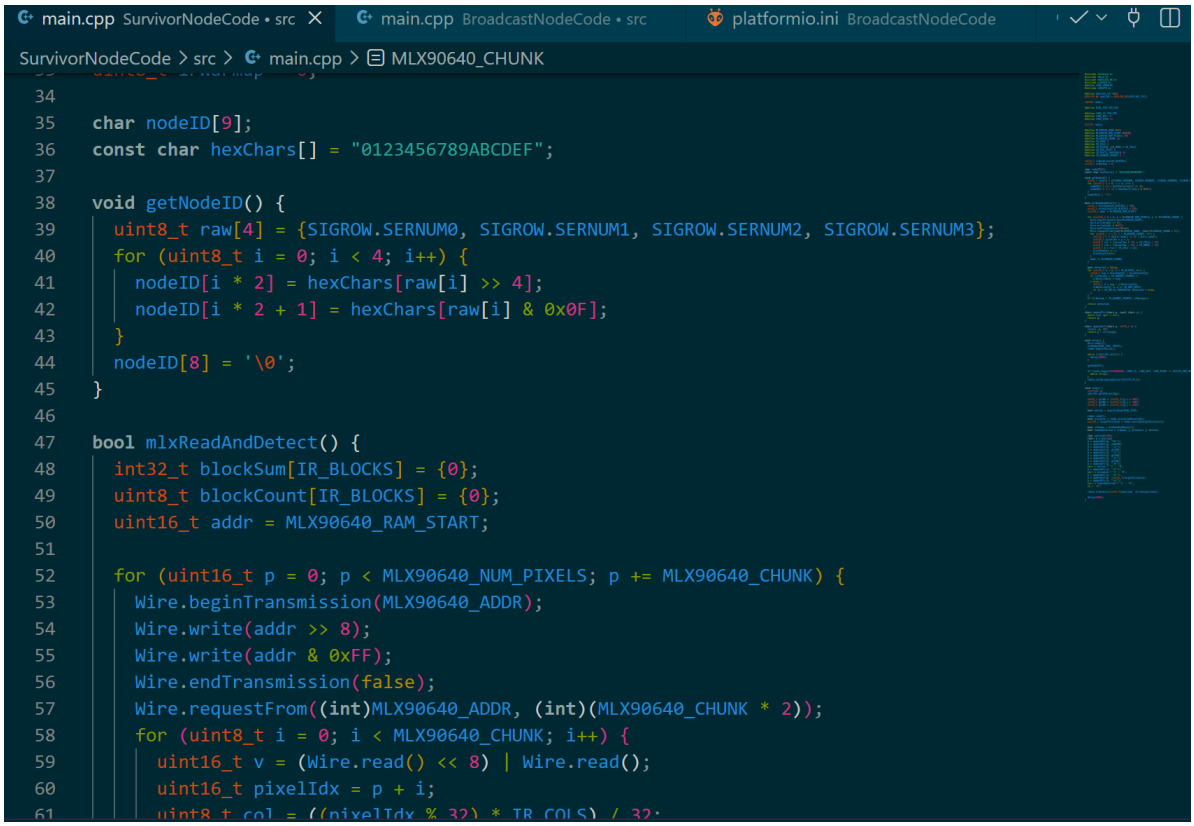
Network Scalability:
FirstResponderSystem utilizes a star topology, so new Survivor and Environmental Nodes can be added to the system without needing a full system reconfiguration or restart. The system runs on a two-second transmit interval and runs at SF9 spreading factor, allowing one broadcast node to receive data from dozens of nodes without slowing down or getting too overloaded.

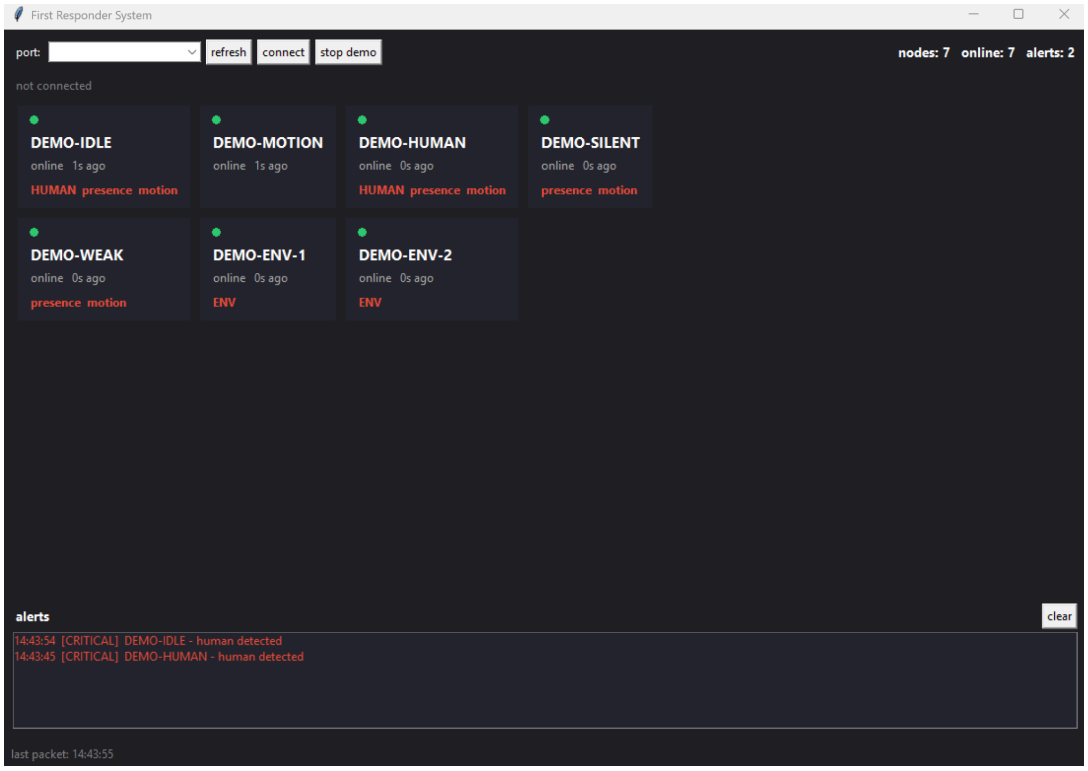
Environmental Impact:
The nodes were programmed to only transmit briefly and infrequently compared to having a constant connection to WiFi networks or cell towers. This way of sending packets allows the nodes to conserve power and maintain a low RF footprint. Since components are soldered, either by stencil or by hand, bad or damaged components can be replaced rather than discarding the whole node.

8. Programming

Firmware Language and Tools Used:
All three nodes were programmed in C++ using PlatformIO Arduino in Visual Studio Code. The code is organized with its own platformio.ini, source code files, and libraries. The code was verified to compile successfully and fit within the boundaries of the flash storage size.

Fitting within flash storage boundaries:
The Survivor and Broadcast nodes run on the ATTINY16-14 MCU, which only has 16 KB of flash. Few vendor libraries were too big to fit when compiled with the LoRa and sensor libraries. A few examples of these libraries included RadioLib (on some nodes) and the MLX90640 library. To combat this issue, the SX1276 library was used instead of RadioLib to directly interface with the chip, and it was lighter since it did not carry configurations for other chips. Similarly, the MLX90640 library was replaced with a custom I2C driver integrated into the code that interfaced with registers to work, rather than pulling full driver libraries like other libraries would do. This method was utilized because disabling specific parts of libraries in platformio.ini was breaking dependencies and library functionality overall, especially for RadioLib. Manual string-building was also used to save flash storage rather than using heavy string libraries.

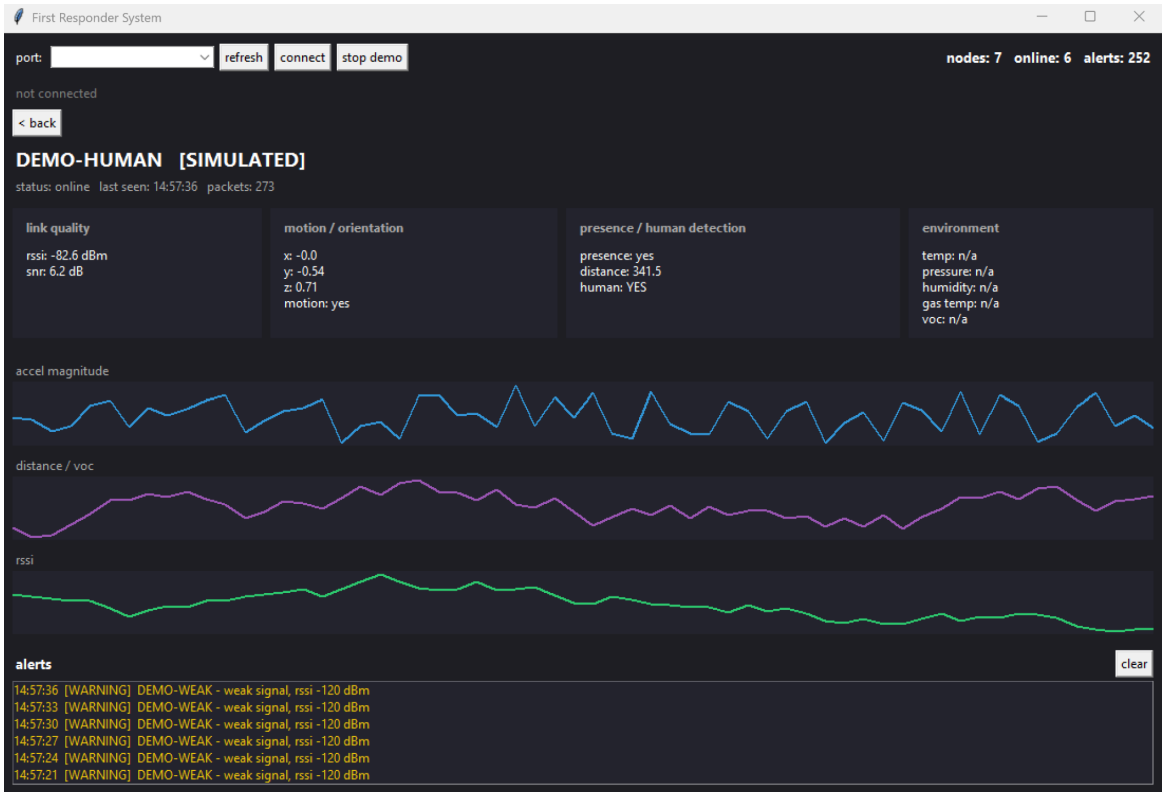




9. User Experience and Design

Computer Software UI:
Since first responders need to interpret data quickly under stressful conditions, the software's user interface (UI) was designed to be human-readable and viewable at-a-glance while still preserving information. Another principle that was used in the creation of the UI was that a first responder should be able to tell the condition and alerts from a node, without needing to parse the sensor values themselves.

At-a-glance Status:
In the UI box of a node, a color-coded status (online, stale, offline) is shown so connection issues can be noticed without reading logs or timestamps. Alerts are also classified with three levels, with info, warning, and critical. This allows a first responder to quickly locate a major issue without digging through a list of other issues or logs.



No Configuration Required:
New nodes are able to automatically join the list in the computer once they are detected. Since information is sent over serial and the nodes utilize a star-topology, there is no need to waste time setting up nodes, naming the nodes, or manually pairing. Nodes can easily be added on-the-fly without needing any special technicians or specialized people, since they automatically generate a name from information in the MCU's fuses.

No-Hardware Preview:
Demo mode in the computer software allows first responders or judges to get familiar with the interface. Demo mode populates the interface with "fake" nodes that send randomly-generated packets to simulate disasters/incidents. This mode can also show the software at events without needing systems or nodes on hand.

10. Future Improvements

Things to be done:

- Data Logging - Save incoming packets into a database (online or locally) for future analysis

- Packet Resending - Make a node resend a packet if it was dropped or information was lost
- Encryption/Auth - Add authentication and encryption to the packets to keep information safe
- GPS - Add GPS to the Survivor and Environmental Nodes so that the nodes show detections on a map, not a list
- Mobile/Web interface - Expand the Tkinter interface to allow first responders to view info from a phone or web browser
- Adjust frequency based on region - Different countries and parts of the world use different frequencies for LoRa/unlicensed bands, so we need to ship different modems for each region of the world
- FCC Part 15 Compliance - Any deployment of the FirstResponderSystem would require compliance with FCC Part 15's transmit power, duty cycle, and frequency hopping regulations

11. Team and Credits

Creator Info:
This project was designed, built, and recorded by Aneesh Lingala.

Acknowledgements (names ending in .h are C or C++ libraries)**:**
- KiCAD for the PCB designing software
- Arduino.h - Arduino Core Team
- Wire.h - Arduino Core Team
- SX1276.h - Matthias Prinke
- ADXL345_WE.h - Wolfgang Ewald
- LD2410.h - Nick Reynolds
- Bosch_BME280_Arduino.h - Frank Häfele
- Zanshin_BME680.h - SV-Zanshin Arduino Org.
- Radiolib.h (Environmental Node) - Jan Gromeš
- WiFi.h (ESP32) - Espressif Systems
- PySerial (computer software) - Chris Liechti
- Tkinter (computer software) - Guido van Rossum and Steen Lumholt

Works Cited:

- Federal Communications Commission, Public Safety and Homeland Security Bureau. (2018). *2017 Atlantic hurricane season: Impact on communications report and recommendations* (Public Safety Docket No. 17-344). https://docs.fcc.gov/public/attachments/DOC-353805A1.pdf
- Augustin, A., Yi, J., Clausen, T., & Townsley, W. (2016). A study of LoRa: Long range & low power networks for the internet of things. *Sensors*, *16*(9), 1466. https://doi.org/10.3390/s16091466